

Sistema Inteligente de Soporte Hospitalario

LaSalle Health Center

MEMORIA TECNICA

Plataforma containerizada de IA y Big Data
para clasificacion radiologica y triaje clinico

Equipo de desarrollo

Oriol Fontcuberta

Unai Canet

Albert Garrido

Asignatura: Sistemes d'Aprenentatge Automatic / Big Data

Fecha de entrega: 2026-05-20

0. Indice

1. Resumen ejecutivo
2. Descripcion del problema
3. Datos
4. Arquitectura del sistema
5. Modelos de Inteligencia Artificial
6. Pipeline de Big Data
7. Backend (API REST)
8. Dashboard
9. Automatizaciones
10. Integraciones
11. Calidad, testing y CI/CD
12. Justificaciones tecnicas y alternativas
13. Diario de desarrollo con IA
14. Reflexion critica
15. Consideraciones eticas y legales
16. Conclusiones
 - A. Anexo: estructura del repositorio
 - B. Anexo: variables de entorno
 - C. Anexo: comandos de ejecucion

1. Resumen ejecutivo

1.1 Vision general del sistema

Este documento describe la plataforma desarrollada para el hospital ficticio **LaSalle Health Center**, una solución integral de soporte clínico que combina técnicas de inteligencia artificial y big data en un entorno containerizado. El sistema cubre el ciclo completo: ingesta de imágenes radiológicas y datos tabulares, validación de calidad, clasificación automática (radiografía y triaje), persistencia dual (PostgreSQL + MinIO), generación de informes y visualización operativa a través de un dashboard web con autenticación.

La arquitectura se ha diseñado siguiendo principios de separación estricta de responsabilidades (Layered Architecture y Domain-Driven Design adaptados a Python), garantizando transaccionalidad ACID en todas las operaciones compuestas y exposición de métricas operativas en formato JSON consumibles por el dashboard. El despliegue se realiza con **un único comando** (*docker compose up -d*) y cubre nueve servicios containerizados coordinados a través de una red Docker común.

1.2 Tecnologías clave

- **Backend:** FastAPI 0.11x con Pydantic, arquitectura en capas (api / services / repositories / db / storage).
- **Dashboard:** Flask con app factory, blueprints, autenticación local con hashes Werkzeug y sesiones firmadas.
- **Base de datos relacional:** PostgreSQL 16.13 (alpine) con connection pool psycopg 2-10.
- **Almacenamiento de objetos:** MinIO compatible con S3 (boto3).
- **Procesamiento distribuido:** Apache Spark 3.5.8 con un master y un worker.
- **Modelos IA:** Random Forest calibrado (traje) y ResNet18 (clasificación radiológica).
- **Orquestación:** Docker Compose con perfiles (*pipeline*, *training*, *triage-training*).
- **Tests y CI:** pytest con dobles que respetan contratos reales y GitHub Actions.

1.3 Estado de la entrega

El proyecto cumple **todos los requisitos mínimos del enunciado** en su alcance técnico: modelo de IA, pipeline Big Data, dos sistemas de almacenamiento, automatizaciones, dashboard, containerización con un solo comando, calidad de datos, logging y documentación. Los dos modelos de IA (Random Forest de triaje y CNN ResNet18 de radiología) están entrenados, persistidos en */models* y cargados en caliente por el backend al iniciar.

- Infraestructura Docker Compose: **9 servicios** operativos con healthchecks y volúmenes persistentes.
- Backend FastAPI modular con **5 routers** (health, triage, radiology, quality, metrics).
- Dashboard Flask modular con app factory, blueprints de autenticación y vistas operativas.
- Pipeline Spark modular separando validación, transformación, escritura y reporte.
- **Dos modelos** de IA entrenados y operativos: triaje Random Forest (accuracy 80.6 % sobre 8.000 muestras) y CNN ResNet18 (accuracy 95.04 % en validación y 93.94 % en test).
- Cobertura de tests amplia: 18 archivos de tests en backend y 6 en dashboard.

- Acceso directo por HTTP a backend (:8000) y dashboard (:8501); HTTPS fuera de alcance del prototipo.

2. Descripcion del problema

2.1 Contexto del hospital LaSalle Health Center

LaSalle Health Center es una organizacion sanitaria de tamano medio en proceso de transformacion digital. Genera diariamente grandes volumenes de datos clinicos y operativos (historias clinicas, registros de pacientes, pruebas diagnosticas, logs de sistemas) que hasta la fecha no se aprovechan sistematicamente para extraer conocimiento, detectar patrones ni automatizar tareas repetitivas.

La direccion del hospital ha contratado al equipo, en su rol de consultora tecnologica especializada en Inteligencia Artificial y Big Data, para disenar e implementar una plataforma que aporte valor real en un entorno sanitario, respetando las restricciones eticas, legales y operativas inherentes al sector.

2.2 Necesidades detectadas

- Extraer conocimiento util de grandes volumenes de datos heterogeneos.
- Detectar patrones clinicos relevantes en imagenes medicas (radiografias de torax).
- Automatizar procesos internos: validacion de datos, generacion de informes, alertas.
- Apoyar la toma de decisiones medicas y operativas con metricas y resultados interpretables.
- Disponer de una infraestructura reproducible, desplegable con un unico comando y portable a cualquier maquina.

2.3 Objetivos del proyecto

Los objetivos concretos planteados son los siguientes:

- Implementar un sistema de **clasificacion triple de radiografias de torax** en las categorias *Sana*, *Neumonia* y *COVID-19*.
- Disponer de un mecanismo de **triaje clinico automatizado** que asigne nivel de riesgo y prioridad a partir de sintomas y signos vitales.
- Disenar un **pipeline Big Data** con Spark que ingiera, valide, transforme y registre lotes de estudios.
- Combinar dos tipos de almacenamiento: **PostgreSQL** para datos estructurados y **MinIO** para datos no estructurados (imagenes, informes JSON).
- Exponer una **API REST** tipada para consumo desde el dashboard y desde clientes externos.
- Ofrecer una **interfaz web** con autentificacion para personal sanitario.
- Implementar **automatizaciones**: generacion de informes, alertas ante eventos relevantes, deteccion de problemas de calidad.
- Garantizar **despliegue reproducible** mediante Docker Compose con un unico fichero de composicion.
- Documentar el proceso de forma profesional, incluyendo SDD, diario de IA y consideraciones eticas.

2.4 Alcance funcional

El proyecto es un **prototipo funcional end-to-end**. Cubre todos los componentes pedidos por el enunciado. La inferencia radiologica se sirve con el **modelo CNN ResNet18 ya entrenado** sobre un dataset publico de radiografias de torax con tres clases (Sana, Neumonia, COVID-19). El artefacto persistido es *models/radiology_cnn_resnet18.pt* y las metricas completas se encuentran en *models/radiology_cnn_metrics.json*.

El triaje clinico se entrega como modelo **Random Forest calibrado** ya entrenado y persistido en *models/triage_model.joblib*, con metricas documentadas en *models/triage_metrics.json*. El backend lo carga al arranque con fallback automatico a un sistema de reglas determinista si el artefacto no esta presente.

3. Datos

3.1 Fuentes de datos

Imágenes de radiografías de tórax (no estructurado)

Las imágenes son los datos no estructurados centrales del sistema. Se aceptan los formatos JPG, JPEG y PNG con un tamaño máximo de 5 MB. La validación se realiza en dos niveles: cliente (Flask) y servidor (FastAPI), comprobando extensión declarada, tipo MIME y contenido binario real mediante Pillow para evitar archivos malformados o tipos engañados.

Para entrenamiento del modelo CNN se prevé la incorporación de un dataset público como el **COVID-19 Radiography Database** (Kaggle, ~21k imágenes) siguiendo la estructura de directorios `data/radiology_dataset/{train,val}/{Sana,Neumonia,COVID-19}/`.

CSV de estudios radiológicos (tabular, lote)

El pipeline Spark procesa lotes desde `data/incoming/radiology_studies.csv` con cabecera y las siguientes columnas obligatorias:

- **study_id**: identificador único del estudio.
- **patient_age**: edad del paciente.
- **patient_sex**: M, F o U.
- **image_object_key**: clave del objeto en MinIO.
- **label**: etiqueta clínica (Sana, Neumonia, COVID-19).
- **acquisition_date**: fecha de adquisición.
- **source**: equipo o departamento origen.

Datos sintéticos para entrenamiento de triaje

El triaje se entrena con un dataset sintético generado por `ml/train_triage.py`, inspirado en los protocolos **MEWS** (Modified Early Warning Score) y **NEWS2** (National Early Warning Score 2) utilizados en hospitales del Reino Unido y España. La generación incluye:

- 8.000 muestras sintéticas con variables continuas (edad, frecuencia cardíaca, saturación, presión, frecuencia respiratoria, temperatura) y binarias (síntomas).
- Etiquetas calculadas con reglas MEWS/NEWS2 acotadas a cuatro niveles: *low*, *medium*, *high*, *critical*.
- Ruido de etiquetado del 6 % introducido deliberadamente para forzar generalización y evitar memorización.
- División 80/20 entre entrenamiento y test, estratificada por clase.

3.2 Proceso de ingesta

- **Ingesta desde dashboard**: el usuario sube una imagen por formulario HTML; Flask valida en cliente y delega al backend.
- **Ingesta desde API**: clientes externos pueden invocar `POST /predict` directamente con la imagen binaria.

- **Ingesta de lotes:** el pipeline Spark se dispara con *docker compose --profile pipeline up pipeline* y lee el CSV indicado por *PIPELINE_INPUT_PATH*.
- **Ingesta para entrenamiento:** el modulo *model-trainer* lee imagenes desde la estructura de directorios esperada en *data/radiology_dataset/*.

3.3 Limpieza y validacion

La validacion se aplica de forma consistente en cada etapa del pipeline:

- **Imagenes:** extension JPG/JPEG/PNG, tamano ≤ 5 MB, MIME real verificado con Pillow, deteccion de coherencia entre extension y formato real (rechazo de JPEG renombrado como PNG).
- **CSV tabular:** comprobacion de columnas obligatorias, validacion de tipos, recorte de espacios en blanco, etiquetas validas, no valores nulos en campos clave.
- **Detection de duplicados:** por *image_object_key*; ambos duplicados se rechazan.
- **Separacion de registros:** validos van a *data/processed/radiology_clean*; rechazados se documentan en el informe del run.
- **Eventos de calidad:** cualquier rechazo genera un evento de severidad *medium* en la tabla *quality_events*; un fallo de pipeline genera evento *high*.

3.4 Transformaciones aplicadas

- **Spark:** normalizacion de strings (trim), conversion de tipos, calculo de columnas derivadas y particionado del DataFrame para escritura.
- **Imagenes (entrenamiento):** resize a 224x224, conversion a tensor, normalizacion con la media y desviacion estandar de ImageNet (*mean*=[0.485, 0.456, 0.406], *std*=[0.229, 0.224, 0.225]).
- **Augmentations en training:** horizontal flip aleatorio ($p=0.5$) y rotacion ± 8 grados para reducir overfitting con datasets pequenos.
- **Imagenes (inferencia ResNet18):** resize a 224x224, conversion a tensor RGB, normalizacion con la misma media y desviacion utilizadas en entrenamiento (parametros de ImageNet) y forward a traves de la red en modo *eval*.

3.5 Almacenamiento final

Tipo de dato	Almacenamiento	Justificacion
Imagenes binarias	MinIO (uploads/)	API S3, no estructurado, gran tamano
Informes radiografia	MinIO (radiology-reports/)	JSON inmutable, asociado a un estudio
Informes triaje	MinIO (triage-reports/)	JSON, persistente, consultable
Informes pipeline	MinIO (pipeline-reports/)	JSON con metricas por ejecucion
Estudios radiologicos	PostgreSQL	Relacional, consultas operativas y agregados
Triajes	PostgreSQL	Historial estructurado
Eventos de calidad	PostgreSQL	Indexable, contadores por severidad
Ejecuciones pipeline	PostgreSQL	Trazabilidad de runs Spark
Datos tabulares procesados	Filesystem (JSON)	Salida intermedia del pipeline

4. Arquitectura del sistema

4.1 Vision general

El sistema sigue una arquitectura de **microservicios containerizados** coordinados por Docker Compose. Cada servicio tiene una responsabilidad unica y se comunica con el resto mediante protocolos estandar (HTTP, S3, PostgreSQL wire protocol, Spark RPC). Todos los servicios comparten una red Docker comun (*hospital_network*) que permite resolver nombres por DNS interno.

```
Browser --> Dashboard (Flask:8501)
          |
          |--> Backend (FastAPI:8000)
          |   |
          |   |--> PostgreSQL (db:5432)
          |   |--> MinIO (minio:9000)
          |   '--> Spark (spark:7077)
          |
          '--> MinIO (uploads directos)

Pipeline (Spark job, on-demand)
--> Lee CSV de /data/incoming
--> Escribe limpio en /data/processed
--> Registra ejecucion en PostgreSQL
--> Publica informe JSON en MinIO

Model-trainer (entrenamiento CNN, on-demand)
--> Lee dataset de /data/radiology_dataset
--> Persiste artefacto en /models
```

4.2 Diseno del pipeline de datos

El pipeline de datos se concibe en cuatro fases claras: **ingesta**, **validacion**, **transformacion** y **servicio**. Cada fase es independiente y testeable, y el pipeline Spark esta modularizado en submodulos (*validators*, *transforms*, *writers*, *reports*) para facilitar mantenimiento.

- **Ingesta:** dashboard (imagenes individuales), API (clientes externos), pipeline Spark (lotes CSV).
- **Validacion:** esquema, tipos, integridad, calidad de imagen.
- **Transformacion:** preprocesamiento de imagen, limpieza tabular, derivacion de campos.
- **Servicio:** API REST tipada, dashboard web, MinIO API S3.

4.3 Infraestructura containerizada

Servicios Docker Compose

Servicio	Imagen	Puerto	Responsabilidad
backend	hospital-backend (Python 3.11)	8000	API REST, inferencia, persistencia
dashboard	hospital-dashboard (Python)	8501	UI web, autenticacion, subida
db	postgres:16.13-alpine	5432	Datos estructurados
minio	minio:RELEASE.2024-01-11	9000 / 9001	Almacenamiento de objetos
spark	spark:3.5.8-python3	interno	Master Spark
spark-worker	spark:3.5.8-python3	interno	Worker Spark (2 cores / 1 GB)
pipeline	hospital-pipeline	perfil pipeline	Job Spark de ingesta y calidad
model-trainer	hospital-model-trainer	perfil training	Entrenamiento CNN
triage-trainer	hospital-triage-trainer	perfil triage-training	Entrenamiento RF triaje

Red Docker

Todos los servicios viven en la red *hospital_network* con driver bridge. Esto permite que el backend referencie PostgreSQL como *db:5432*, MinIO como *minio:9000* y Spark como *spark://spark:7077* sin necesidad de exponer puertos al host. Los puertos publicados (backend, dashboard, MinIO, db) se enlazan a *127.0.0.1* para evitar exposicion en la red local.

Volumenes persistentes

- **postgres_data**: persistencia de las tablas relacionales.
- **minio_data**: persistencia de los buckets de objetos.
- **./data** (bind mount): datos de ingesta y procesados compartidos entre pipeline y model-trainer.
- **./models** (bind mount, read-only en backend): artefactos de modelos compartidos.

Perfiles de despliegue

Docker Compose soporta perfiles para encender servicios opcionales sin interferir con el arranque por defecto. Esto separa cargas pesadas del flujo habitual:

- **Sin perfil**: backend, dashboard, db, minio, spark, spark-worker.
- **--profile pipeline**: lanza el job Spark de ingesta.
- **--profile training**: entrena el modelo CNN ResNet18 sobre el dataset radiografico.
- **--profile triage-training**: entrena el Random Forest de triaje.

4.4 Relacion entre componentes

El flujo end-to-end de un estudio radiologico atraviesa cinco fronteras de integracion:

- **1. Navegador --> Dashboard:** HTTP con cookie de sesion firmada (SECRET_KEY), HttpOnly, SameSite=Lax.
- **2. Dashboard --> MinIO:** subida directa de la imagen a *uploads/<uuid>.<ext>* mediante boto3.
- **3. Dashboard --> Backend:** POST */predict* con la imagen y la clave de objeto ya conocida.
- **4. Backend --> Servicios internos:** inferencia, persistencia en PostgreSQL (pool) y en MinIO (informe JSON), todo en una sola transaccion ACID.
- **5. Respuesta:** el dashboard renderiza resultado, probabilidades, banderas de calidad y enlace al informe.

4.5 Capa de seguridad

- **Autenticacion local:** usuarios admin y doctor con contraseñas hash Werkzeug (PBKDF2-SHA256, 1M iteraciones).
- **Roles:** admin ve almacenamiento MinIO y todas las metricas; doctor accede a radiologia y resultados.
- **Sesiones:** cookie firmada con SECRET_KEY de longitud ≥ 32 bytes; expira al cerrar el navegador.
- **Puertos restringidos:** todos los puertos publicados estan enlazados a 127.0.0.1, no a 0.0.0.0.
- **Variables sensibles:** externalizadas en *.env* con la sintaxis *\${VAR:?Set VAR in .env}* que falla rapido si faltan.
- **Credenciales nunca en repositorio:** *.env*, certificados y artefactos sensibles excluidos por *.gitignore*.

5. Modelos de Inteligencia Artificial

5.1 Modelo de triaje (Random Forest calibrado)

Justificacion de la arquitectura

El triaje clinico opera sobre datos tabulares de baja dimensionalidad (~17 variables tras one-hot encoding de sintomas). Para este tipo de problema se ha seleccionado un **Random Forest** con 400 arboles y profundidad maxima 12, posteriormente **calibrado isotonicamente** con `CalibratedClassifierCV` para producir probabilidades clinicamente interpretables. La eleccion se justifica por:

- **Interpretabilidad:** feature importances directamente comparables con la intuicion clinica (saturacion y bleeding son las variables top).
- **Robustez frente a outliers:** característica intrínseca de los ensembles de arboles.
- **Sin necesidad de GPU:** entrenable en CPU en menos de 20 segundos.
- **Buen comportamiento con datasets pequenos:** 8.000 muestras son suficientes para generalizar.
- **Calibracion isotonica:** corrige el sesgo de probabilidades hacia 0.5/1 tipico en RF y produce probabilidades clinicamente fiables.

Generacion del dataset sintetico

El script `ml/train_triage.py` genera 8.000 muestras a partir de reglas inspiradas en MEWS y NEWS2. Cada muestra incluye seis variables continuas (edad, frecuencia cardiaca, saturacion, presion sistolica, frecuencia respiratoria y temperatura) y diez variables binarias correspondientes a sintomas frecuentes (dolor toracico, disnea, perdida de conciencia, fiebre, tos, dolor abdominal, cefalea, mareo, vomitos, sangrado).

Las etiquetas se calculan con un sistema de puntuacion que respeta la logica clinica: hipoxia, taquicardia e hipotension contribuyen al score, los sintomas red-flag (dolor toracico, disnea, perdida de conciencia, sangrado) elevan el riesgo, y el resultado se discretiza en cuatro niveles. Se anade un **6 % de ruido** en las etiquetas para forzar la generalizacion.

Entrenamiento y calibracion isotonica

- Train/test split estratificado 80/20 con `random_state=42`.
- Entrenamiento del Random Forest con 400 arboles y `max_depth=12`.
- Calibracion isotonica sobre las probabilidades del clasificador base.
- Persistencia del modelo calibrado y de las metricas en `models/triage_model.joblib` y `models/triage_metrics.json`.
- Duracion total del entrenamiento: ~16 segundos en CPU.

Metricas y resultados

Metrica	low	medium	high	critical	Macro
Precision	0.83	0.74	0.69	0.97	0.81
Recall	0.79	0.84	0.58	0.92	0.78
F1-score	0.81	0.79	0.63	0.94	0.79
Soporte	363	583	258	396	1.600

La **accuracy global es 80.6 %** sobre 1.600 muestras de test. La clase *critical* alcanza una precision de 0.97 y un recall de 0.92, que es exactamente lo deseable clinicamente: minimizar falsos negativos en los casos mas graves. La clase *high* es la mas dificil (recall 0.58), debido a que se solapa con *medium* en la frontera; este punto se documenta como limitacion en la seccion 14.

5.2 Modelo CNN ResNet18 (clasificacion triple de radiografias)

Justificacion de la arquitectura

Para la clasificacion de radiografias se ha seleccionado **ResNet18**. La justificacion responde a cuatro criterios:

- **Madurez probada:** ResNet18 es uno de los backbones mas usados en imagen medica con literatura abundante.
- **Bajo coste de inferencia:** 11.7 M parametros, latencia de inferencia baja incluso en CPU.
- **Profundidad razonable:** 18 capas suficientes para capturar patrones radiologicos sin sobreparametrizar.
- **Escalabilidad:** si se necesita mas capacidad se puede pasar a ResNet50 o EfficientNet sin cambiar el resto del pipeline.

Configuracion del entrenamiento

El modelo se ha entrenado en GPU (CUDA) sobre un dataset publico de radiografias de torax con tres clases. La configuracion final usada en la version 0.2.0 del artefacto se resume en la siguiente tabla.

Parametro	Valor
Arquitectura	ResNet18 (entrenada desde cero)
Dispositivo	CUDA (GPU)
Epochs	5
Batch size	16
Learning rate	5e-4
Optimizador	AdamW
Loss	CrossEntropyLoss con class weights
Class weights	COVID-19=3.73, Sana=1.35, Neumonia=0.50
Val split	20 % estratificado

Parametro	Valor
Seed	42 (reproducibilidad)
Duracion total	1.055 segundos (~17.5 minutos)

Los **class weights** son inversamente proporcionales a la frecuencia de cada clase. El dataset esta fuertemente desbalanceado (Neumonia es ~7 veces mas frecuente que COVID-19), y esta ponderacion fuerza al modelo a no ignorar las clases minoritarias durante el entrenamiento.

Preprocesamiento y data augmentation

Etapas	Operacion
Resize	224 x 224
Normalizacion	mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
Augmentation (train)	RandomHorizontalFlip(p=0.5), RandomRotation(8 grados)
Conversion	ToTensor

Distribucion del dataset

Particion	COVID-19	Sana	Neumonia	Total
Train	368	1.013	2.734	4.115
Validation	92	253	684	1.029
Test	116	317	855	1.288
Total	576	1.583	4.273	6.432

Evolucion del entrenamiento por epoch

Epoch	Train loss	Train acc	Val loss	Val acc
1	0.402	0.823	0.221	0.915
2	0.290	0.877	0.321	0.865
3	0.226	0.903	0.200	0.930
4	0.206	0.910	0.274	0.894
5	0.214	0.907	0.147	0.950

El mejor checkpoint corresponde a la epoch 5, con una accuracy de validacion del **95.04 %** y la loss mas baja observada. La curva muestra cierta fluctuacion en validacion (caida en epoch 2 y 4) característica del entrenamiento desde cero sin warmup; la tendencia general es claramente ascendente.

Resultados sobre el conjunto de validacion (mejor checkpoint)

Accuracy global: 95.04 % — Loss: 0.147

Clase	Precision	Recall	F1-score	Soporte
COVID-19	0.899	0.967	0.932	92
Sana	0.901	0.937	0.919	253
Neumonia	0.978	0.953	0.965	684
Macro avg	0.926	0.952	0.939	1.029
Weighted avg	0.952	0.950	0.951	1.029

Matriz de confusion sobre validacion

Real / Predicho	COVID-19	Sana	Neumonia
COVID-19	89	1	2
Sana	3	237	13
Neumonia	7	25	652

Resultados sobre el conjunto de test (generalizacion)

Accuracy global: 93.94 % — Loss: 0.168

Clase	Precision	Recall	F1-score	Soporte
COVID-19	0.958	0.983	0.970	116
Sana	0.864	0.905	0.884	317
Neumonia	0.967	0.946	0.956	855
Macro avg	0.930	0.945	0.937	1.288
Weighted avg	0.941	0.939	0.940	1.288

Matriz de confusion sobre test

Real / Predicho	COVID-19	Sana	Neumonia
COVID-19	114	2	0
Sana	2	287	28
Neumonia	3	43	809

Lectura clinica clave: en el conjunto de test, el modelo detecta correctamente **114 de 116 casos de COVID-19** (recall 0.983). Los 2 falsos negativos se clasifican como *Sana*, pero **ninguno como Neumonia**: el modelo no confunde COVID con neumonia comun, que es exactamente la propiedad mas valiosa clinicamente para evitar aislamientos tardios.

5.3 Servicio de inferencia y heurística de respaldo del triaje

El backend incorpora dos estrategias de servicio segun el modelo: para radiologia, *InferenceService* carga el checkpoint ResNet18 al arrancar y exige su presencia (sin *.pt*, el contenedor falla rapido para evitar predicciones falsas con cualquier sustituto improvisado). Para triaje, *triage_model* sigue un patron singleton con *threading.Lock* que carga *triage_model.joblib* bajo demanda en la primera prediccion.

El servicio de triaje ademas conserva una **heurística de reglas** (*_baseline* en *triage_service.py*) que se activa unicamente cuando el Random Forest no esta disponible (artefacto ausente, version incompatible o error de carga). Esta heurística calcula un score basado en frecuencia cardiaca, saturacion, presion sistolica y sintomas dominantes (dolor toracico, disnea, perdida de consciencia) y lo mapea a uno de los cuatro niveles de riesgo. Su rol es estrictamente de **safety net**: en operacion normal el flujo invoca siempre al modelo entrenado.

Tres modelos coexisten en el sistema: (1) ResNet18 para radiografia, (2) Random Forest calibrado para triaje y (3) heurística de reglas como fallback unicamente del triaje. En produccion normal corren los dos modelos entrenados; la heurística queda como salvaguarda y se identifica en las respuestas con el *model_name hospital-triage-rules-fallback*.

5.4 Analisis clinico (el porque)

Tipos de error y matriz de confusion

Para el triaje, la matriz de confusion del Random Forest sobre el test set (1.600 muestras) revela el siguiente comportamiento:

Real / Predicho	low	medium	high	critical
low	286	75	2	0
medium	58	488	36	1
high	2	94	150	12
critical	0	2	28	366

Lectura clinica: los errores se concentran en confusiones entre niveles adyacentes (low <-> medium, medium <-> high). Las confusiones **saltando dos niveles son raras**: solo 2 casos de *low* clasificados como *high* y ninguno como *critical*. Esta es la propiedad mas deseable: el modelo nunca "olvida" un caso critico clasificandolo como trivial.

Impacto de falsos negativos

- **Falso negativo critical:** un paciente critico clasificado como menor riesgo es el peor escenario. En el test: 0 casos de critical clasificados como low, 2 como medium. **Recall critical = 0.92**.
- **Falso negativo high:** 96 casos de high clasificados como medium o low (recall 0.58). Clinicamente, esto exige un proceso de revision humana para todos los casos *medium* con flags clinicas.
- **Falso negativo COVID-19 (CNN):** con recall 0.983 sobre test, el CNN solo erra 2 de 116 COVID, y ambos como *Sana* (ninguno como Neumonia). Toda prediccion COVID-19 dispara automaticamente un evento de severidad alta en *quality_events*.

Limitaciones tecnicas reales

- El triaje opera con seis variables vitales y diez sintomas; no contempla historia clinica, comorbilidades o resultados analiticos.
- El dataset de triaje es sintetico; un despliegue real requeriria reentrenamiento con datos hospitalarios.
- El CNN se ha entrenado solo 5 epochs sobre 4.115 imagenes; un despliegue clinico requeriria mas datos, validacion externa y revision por radiologos.
- El sistema asume tres clases radiologicas; cualquier patologia distinta sera clasificada incorrectamente.

6. Pipeline de Big Data

6.1 Justificacion de Apache Spark

Se ha elegido **Apache Spark 3.5.8** con PySpark como motor de procesamiento distribuido. Las razones se documentan en detalle en el capitulo 12, pero resumidamente: Spark es el estandar de facto en entornos hospitalarios y de investigacion biomedica, ofrece API SQL madura, soporta escalado horizontal transparente y se integra de forma trivial con almacenamientos compatibles con S3 (como MinIO).

6.2 Fases del pipeline

El job Spark *radiology_pipeline.py* cubre cuatro fases secuenciales:

- **1. Ingesta:** lectura del CSV desde *PIPELINE_INPUT_PATH* usando *spark.read.csv()* con cabecera, inferencia de tipos y opcion *multiLine*.
- **2. Validacion:** comprobacion de columnas obligatorias, tipos, etiquetas validas y deteccion de duplicados por *image_object_key*.
- **3. Transformacion:** trim de strings, normalizacion de campos categoricos, recalcu de columnas derivadas.
- **4. Escritura:** persistencia de datos limpios en JSON a *PIPELINE_OUTPUT_PATH*, registro de ejecucion en PostgreSQL y publicacion de informe JSON en MinIO.

6.3 Validators, transforms y writers

El pipeline esta **modularizado** en submodulos especializados, cada uno con una responsabilidad unica:

Modulo	Archivo	Responsabilidad
validators	csv_schema.py	Schema obligatorio, validacion de etiquetas y nulos
transforms	quality_filter.py	Filtrado de registros invalidos, deteccion duplicados
writers	postgres_writer.py	Inserciones idempotentes en PostgreSQL
writers	minio_writer.py	Subida de informes JSON a MinIO
reports	run_report.py	Construccion del informe por ejecucion
core	spark_factory.py	Factory de SparkSession con configuracion homogenea
core	config.py	Carga tipada de variables de entorno
core	logging.py	Configuracion JSON estructurada para logs

6.4 Persistencia de ejecuciones y reportes

Cada ejecucion del pipeline genera:

- Una fila en la tabla **pipeline_runs** con timestamp, parametros, status y duracion.

- Un informe JSON en MinIO bajo **pipeline-reports/<run_id>.json** con: distribución de clases en registros aceptados, muestra de hasta 25 registros rechazados, lista de claves duplicadas y resumen estadístico.
- Eventos en **quality_events** si hay rechazos (severidad *medium*) o si el job falla (severidad *high*).
- Logs estructurados JSON visibles via *docker compose logs pipeline*.

El dashboard consulta el endpoint *GET /metrics* del backend para mostrar el estado de la última ejecución (timestamp, status, contadores de procesados, rechazados y duplicados). Esto cierra el bucle de visibilidad operativa.

7. Backend (API REST)

7.1 Estructura modular

El backend FastAPI sigue una **arquitectura en capas** con cinco responsabilidades claramente separadas:

```
backend/app/
|- main.py                app factory, lifespan, middleware
|- api/
|   |- deps.py            inyeccion de dependencias (lru_cache)
|   |- errors.py          handlers de error globales
|   '- routers/          5 routers HTTP
|       |- health.py
|       |- triage.py
|       |- radiology.py
|       |- quality.py
|       '- metrics.py
|- services/              logica de negocio
|   |- inference_service.py carga y forward del ResNet18
|   |- triage_service.py   orquestacion triaje
|   |- triage_model.py     carga del Random Forest
|   |- radiology_service.py flujo completo de prediccion
|   |- quality_service.py  gestion de eventos
|   |- metrics_service.py  agregados para dashboard
|   '- health_service.py   comprobacion de dependencias
|- repositories/          acceso a datos (psycpg)
|   |- radiology_repository.py
|   |- triage_repository.py
|   |- quality_repository.py
|   '- metrics_repository.py
|- db/                    session, pool, schema, transaccion
|- storage/               adaptador S3 (boto3)
|- schemas/               Pydantic (DTOs tipados)
|- core/                  config tipada, logging
'- utils/                 helpers transversales
```

7.2 Endpoints expuestos

Metodo y ruta	Proposito
GET /health	Healthcheck de dependencias
POST /triage	Evaluar triaje a partir de sintomas y vitales
GET /triage/history	Listar historial de triajes
GET /triage/{id}	Detalle de un triaje
GET /triage/{id}/report	Informe JSON del triaje en MinIO
POST /predict	Clasificar radiografia
GET /studies/history	Listar historial de estudios radiologicos
GET /studies/{id}	Detalle de un estudio
GET /studies/{id}/report	Informe JSON del estudio
GET /quality/events	Listar eventos de calidad
PATCH /quality/events/{id}/resolve	Marcar evento como resuelto
GET /metrics	Snapshot agregado en JSON consumido por el dashboard

7.3 Capa de persistencia

El backend utiliza un **connection pool** de `psycopg` con tamaño configurable (por defecto 2 a 10 conexiones). El pool se abre en el *lifespan* de FastAPI y se cierra al apagarse. Cada operación compuesta se ejecuta dentro de una **transacción ACID** mediante un context manager que comparte el cursor entre los repositorios implicados, garantizando que el estudio, los eventos asociados y los informes JSON queden persistidos atómicamente.

El esquema de PostgreSQL incluye las tablas *radiology_studies*, *triage_assessments*, *quality_events* y *pipeline_runs*, creadas automáticamente al arranque con reintentos espaciados 2 segundos para tolerar arranques concurrentes con la base de datos.

7.4 Healthchecks y observabilidad

- **Liveness:** *GET /health* responde 200 si el servicio está arriba.
- **Readiness:** verifica conectividad con PostgreSQL y MinIO; devuelve estado por dependencia.
- **Healthcheck Docker:** cada 10 segundos `urllib` comprueba *http://localhost:8000/health*.
- **Logs estructurados:** formato JSON con campos `timestamp`, `level`, `service`, `message` y `trace context` cuando aplica.
- **Snapshot de negocio:** *GET /metrics* retorna agregados consumidos por el dashboard (radiografías procesadas, distribución por clase, triajes por nivel, eventos abiertos por severidad, última ejecución del pipeline).

8. Dashboard

8.1 Arquitectura Flask

El dashboard utiliza Flask con el patron **app factory** y **blueprints**. Esto permite separar la creacion de la aplicacion de su configuracion, facilita el testing con clientes efimeros y permite registrar grupos de rutas como modulos independientes.

```
dashboard/app/
|- factory.py          create_app() con config tipada
|- auth/              Werkzeug, sesiones firmadas, roles
|- blueprints/
|   |- auth.py        /login, /logout
|   '- main.py        /, /upload, /history, /events, /storage
|- services/          cliente HTTP del backend, cliente MinIO
|- presenters/        traduccion de respuestas API a vistas
|- triage/            servicios y formularios de triaje
|- validation/        validacion de imagenes en cliente
|- core/              config tipada, logging JSON
'- templates / static  Jinja2, CSS, JS minimo
```

8.2 Autenticacion y roles

- **Hash de contraseñas:** PBKDF2-SHA256 con 1.000.000 iteraciones (Werkzeug).
- **Variables sensibles:** `ADMIN_USERNAME`, `ADMIN_PASSWORD_HASH`, `USER_USERNAME`, `USER_PASSWORD_HASH` definidas en `.env`.
- **Dos roles:** *admin* (acceso a almacenamiento MinIO y todas las vistas) y *user* (solo radiologia y resultados).
- **Cookie de sesion:** firmada con `SECRET_KEY` (≥ 32 bytes), `HttpOnly`, `SameSite=Lax`, `Secure` en produccion (controlable con `SESSION_COOKIE_SECURE`).
- **Proteccion CSRF:** tokens generados por sesion para formularios criticos.
- **Decorator *login_required*:** aplicado a todas las rutas que no son `/login`.

8.3 Vistas principales

Ruta	Plantilla	Funcion
GET /login	login.html	Formulario de autenticacion
GET /	index.html	Dashboard principal con metricas y atajos
GET /upload	upload.html	Formulario de subida de radiografia
POST /upload	result.html	Resultado de la clasificacion
GET /triage	triage.html	Formulario de evaluacion de triaje
POST /triage	triage_result.html	Resultado del triaje
GET /history	history.html	Historial de estudios y triajes
GET /events	events.html	Eventos de calidad abiertos
GET /storage	storage.html	(admin) consola de MinIO embebida
GET /logout	-	Invalidacion de sesion

8.4 Integracion con backend y MinIO

- **Cliente HTTP del backend:** servicio interno con timeout, manejo de errores y mapeo de respuestas a presenters.
- **Cliente MinIO con boto3:** subida directa de imagenes desde el navegador a `uploads/<uuid>.<ext>`.
- **Visualizacion del resultado:** imagen original incrustada en base64, probabilidades por clase, banderas de calidad y enlace al informe JSON en MinIO.
- **Historico:** paginacion y ordenacion descendente por timestamp.
- **Estado del pipeline:** el dashboard consume `GET /metrics` y renderiza un panel con la ultima ejecucion del job Spark.

9. Automatizaciones

9.1 Creacion automatica de bucket MinIO

El adaptador *ObjectStorage* incluye un metodo *ensure_bucket()* que crea el bucket si no existe, controlado por la bandera *MINIO_AUTO_CREATE_BUCKET*. Esto elimina la friccion de tener que preparar el bucket manualmente antes del primer arranque.

9.2 Generacion automatica de informes JSON

- **Por estudio radiologico:** cada llamada a */predict* persiste un JSON en *radiology-reports/<study_id>.json* con resultado, probabilidades, metadatos y nota clinica.
- **Por triaje:** cada llamada a */triage* persiste un JSON en *triage-reports/<triage_id>.json* con score, nivel de riesgo, prioridad recomendada y desglose por variable.
- **Por ejecucion de pipeline:** cada run del job Spark publica un JSON en *pipeline-reports/<run_id>.json* con distribucion de clases, rechazos y duplicados.

9.3 Eventos de calidad automaticos

Los eventos se generan dentro de la misma transaccion que la operacion que los dispara, garantizando consistencia ACID. Si la operacion falla, los eventos no quedan huérfanos.

Trigger	Severidad	Origen
Prediccion COVID-19	high	Backend, en /predict
Confianza inferior a 0.55	medium	Backend, en /predict
Banderas de calidad de imagen	medium	Backend, en /predict
Registros rechazados por pipeline	medium	Pipeline Spark
Pipeline failure	high	Pipeline Spark
Triage critico	high	Backend, en /triage

9.4 Procesamiento batch del pipeline

- El job Spark se ejecuta bajo demanda con *docker compose --profile pipeline up pipeline*.
- Procesa todo el CSV de entrada en una sola pasada, validando, transformando, escribiendo y reportando.
- El resultado queda inmediatamente accesible para el dashboard via las metricas del backend.
- Cualquier fallo del job genera evento *high* automaticamente y se registra en *pipeline_runs* con status *failed*.

9.5 Healthchecks y reinicios

- **Backend:** healthcheck con *urllib* cada 10 segundos contra */health*; reinicio automatico si falla.
- **PostgreSQL:** healthcheck con *pg_isready* cada 10 segundos.

- **MinIO**: healthcheck con *mc ready local* cada 10 segundos.
- **restart: always** en backend, dashboard, db, minio, spark, spark-worker: reinicio automatico ante cualquier caída.
- **restart: "no"** en pipeline y trainers: se ejecutan una sola vez (jobs batch).
- **depends_on con condition**: el backend solo arranca cuando db y minio estan saludables.

10. Integraciones

10.1 Flujo completo de datos end-to-end

El recorrido completo de un estudio radiológico, desde que el usuario sube la imagen hasta que el informe queda persistido y disponible para consulta, atraviesa los siguientes pasos:

1. Usuario autenticado abre /upload en dashboard
2. Dashboard valida formato/tamaño en cliente (JS)
3. Dashboard sube imagen a MinIO bajo uploads/<uuid>.<ext>
4. Dashboard hace POST /predict al backend con imagen + object_key
5. Backend valida imagen con Pillow (MIME real, contenido)
6. Backend ejecuta InferenceService.predict()
7. Backend abre transacción (Unit of Work)
 - 7.1. INSERT en radiology_studies (PostgreSQL)
 - 7.2. PUT en radiology-reports/<id>.json (MinIO)
 - 7.3. INSERT en quality_events si COVID o baja confianza (PostgreSQL)
 - 7.4. COMMIT atómico
8. Backend devuelve respuesta JSON tipada
9. Dashboard renderiza resultado en result.html
10. Usuario puede descargar el informe desde el enlace MinIO

10.2 Conexiones entre módulos

Origen	Destino	Protocolo	Propósito
dashboard	backend	HTTP / REST	Solicitar predicción / triaje, leer métricas
dashboard	minio	S3 (boto3)	Subida directa de imágenes
backend	db	psycopg / pool 2-10	Persistir estudios, triajes, eventos
backend	minio	S3 (boto3)	Subir informes JSON
backend	modelo (joblib)	filesystem (read-only)	Cargar Random Forest de triaje
pipeline	spark	Spark RPC	Ejecutar el job distribuido
pipeline	db	psycopg	Registrar ejecuciones y eventos
pipeline	minio	S3 (boto3)	Publicar informes de ejecución

10.3 Variables de entorno y configuración externalizada

La configuración se carga de forma **tipada al arranque** (fail-fast). Si una variable obligatoria falta, el servicio aborta inmediatamente con un mensaje claro. La sintaxis `${VAR:?Set VAR in .env}` en `docker-compose.yml` fuerza este comportamiento.

Las variables se agrupan lógicamente en cinco bloques: PostgreSQL, MinIO, autenticación del dashboard, parámetros de entrenamiento y configuración del pipeline. El detalle completo se documenta en el **Anexo B**.

11. Calidad, testing y CI/CD

11.1 Cobertura de tests

El proyecto incluye una suite de tests automatizados organizada en dos capas (unit y api/web). La estrategia es usar **dobles que respetan los contratos reales** de las interfaces, no MagicMock genericos, lo que permite que los cambios de infraestructura (por ejemplo migracion a connection pool) no rompan los tests.

Componente	Archivos de test	Tipo
Backend - unit	12 archivos	Services, config, validacion, logging
Backend - api	6 archivos	FastAPI TestClient con dependency_overrides
Dashboard - unit	4 archivos	Servicios, validacion, presenters, auth
Dashboard - web	2 archivos	Flask test client con blueprints
Total	24 archivos de test	Cubriendo logica de negocio

Las suites se ejecutan con `pytest --cov=app --cov-report=term` en cada modulo. La cobertura de la logica de negocio supera el 90 % en el backend.

11.2 Validacion de calidad de datos

- **En el cliente:** JavaScript verifica extension y tamano antes de la subida.
- **En el servidor (FastAPI):** Pillow comprueba el contenido binario real, no solo el MIME declarado.
- **En el pipeline (Spark):** validacion de schema, deteccion de duplicados, etiquetas validas, no nulos en campos clave.
- **Generacion automatica de eventos:** cualquier registro rechazado o problema de calidad genera una entrada en `quality_events`.
- **Dashboard de calidad:** vista dedicada en `/events` que lista eventos abiertos por severidad.

11.3 Logging centralizado

Todos los servicios (backend, dashboard, pipeline) utilizan el modulo `logging` de Python configurado en formato JSON estructurado. Cada linea de log incluye timestamp ISO 8601, level, servicio, mensaje y contexto adicional (`request_id`, `study_id`, `run_id` cuando aplica). Esto facilita el agregado en herramientas de log management futuras (ELK, Loki).

11.4 Pipeline CI (GitHub Actions)

- Workflow `.github/workflows/ci.yml` dispara en cada push y pull request.
- Ejecuta `pytest --cov` sobre backend y dashboard.
- Valida la sintaxis de `docker-compose.yml` con `docker compose config`.
- Falla rapido ante cualquier fallo de test o de validacion.
- Garantiza que la rama principal mantiene siempre los tests en verde.

12. Justificaciones tecnicas y alternativas

12.1 Eleccion de PostgreSQL + MinIO

Alternativas evaluadas: MySQL, MongoDB, SQLite, GridFS, sistema de archivos local, Amazon S3, Azure Blob.

Decision: PostgreSQL 16 + MinIO.

Motivo: PostgreSQL ofrece transacciones ACID solidas, JSONB para datos semiestructurados, indices parciales y agregados condicionales que simplifican consultas de metricas. MinIO ofrece la API S3 estandar, permitiendo migrar a AWS S3 real con un simple cambio de configuracion. MongoDB se descarto porque la mayoría de datos tienen esquema estable y se benefician de garantias relacionales. GridFS fragmentaria el almacenamiento entre dos sistemas distintos (estructurado y binario en Mongo) complicando los backups.

12.2 Eleccion de Spark sobre Dask y Beam

Alternativas evaluadas: Dask, Apache Beam, pandas con procesamiento por lotes.

Decision: Apache Spark 3.5.8 con PySpark.

Motivo: Spark es el estandar de facto en entornos hospitalarios y de investigacion biomedica con grandes volúmenes. Dask es atractivo para workloads numpy-centric pero menos maduro para SQL y joins. Apache Beam ofrece portabilidad multi-runner que excede el alcance del prototipo. Aunque el volumen del prototipo es pequeño, la eleccion de Spark prepara la plataforma para escalado horizontal real.

12.3 Eleccion de Random Forest para triaje

Alternativas evaluadas: Logistic Regression, Gradient Boosting (XGBoost, LightGBM), red neuronal tabular (MLP), reglas deterministas puras.

Decision: Random Forest con 400 arboles y calibracion isotonica.

Motivo: la interpretabilidad clinica es obligatoria en triaje (cada decision debe poder explicarse a un facultativo). Random Forest ofrece feature importances directamente comparables con la intuicion clinica. XGBoost habria dado quizas un 1-2 % mas de accuracy pero a costa de complejidad de tuning y explicabilidad. Las redes neuronales tabulares no estan justificadas para 17 features con 8.000 muestras. La calibracion isotonica corrige el sesgo de probabilidades tipico en RF.

12.4 Eleccion de ResNet18 para clasificacion radiologica

Alternativas evaluadas: CNN propia, ResNet50, EfficientNet-B0, Vision Transformer (ViT), DenseNet121.

Decision: ResNet18 con transfer learning opcional desde ImageNet.

Motivo: balance entre capacidad (11.7M parametros) y coste de inferencia (latencia < 50 ms en CPU). Literatura medica abundante con este backbone. Transfer learning eficaz incluso con datasets hospitalarios pequenos. ResNet50 anadiria 14M parametros sin ganancia clara en datasets pequenos. ViT requiere muchos mas datos para entrenar desde cero. EfficientNet es competitiva pero menos estandar en literatura clinica.

12.5 Eleccion de FastAPI + Flask

Alternativas para backend: Flask, Django REST, Litestar.

Decision backend: FastAPI.

Motivo: tipado fuerte mediante Pydantic, generacion automatica de OpenAPI consumible desde el dashboard de pruebas, soporte asincrono nativo, ecosistema maduro de inyeccion de dependencias. Flask habria requerido capas adicionales (Marshmallow, Flask-RESTX) para alcanzar la misma validacion.

Alternativas para dashboard: Streamlit, Dash, FastAPI con Jinja, SPA (React/Vue).

Decision dashboard: Flask con Jinja y blueprints.

Motivo: Streamlit no permite personalizacion visual suficiente para una UI medica. Dash esta orientado a visualizacion analitica, no a flujos transaccionales con autentificacion. Un SPA habria sobredimensionado el proyecto.

12.6 Eleccion de Docker Compose

Alternativas evaluadas: Kubernetes (kind / minikube), Nomad, podman-compose.

Decision: Docker Compose.

Motivo: el enunciado pide explicitamente que cualquier persona pueda levantar el sistema con un unico comando. Kubernetes anadiria una curva de aprendizaje desproporcionada para un prototipo. La migracion futura a Kubernetes es factible reutilizando los Dockerfiles existentes.

12.7 Alternativas descartadas

- **SQLAlchemy ORM:** descartado en favor de psycopg directo + repositorios; el proyecto no necesita el overhead de un ORM completo y SQL puro es mas predecible para consultas analiticas.
- **Conexion por request:** descartado en favor de psycopg_pool para evitar el coste de handshake TLS y autentificacion en cada peticion.
- **pgbouncer externo:** descartado por anadir un servicio mas; el pool del cliente es suficiente para la carga del prototipo.
- **Pipelines schedulados con cron:** descartado para esta entrega; el pipeline se ejecuta on-demand. Migracion futura a Airflow o Prefect documentada en el capitulo 14.
- **Telemetria externa (Datadog, New Relic):** descartado por preservar control de datos sensibles y reducir dependencia de SaaS de terceros.

13. Diario de desarrollo con IA

13.1 Herramientas utilizadas y justificacion

El proyecto se ha desarrollado utilizando de forma combinada varias herramientas de IA generativa, cada una con un proposito definido:

- **Claude Code** (Anthropic, Claude Opus 4.7): utilizado por el equipo para refactor de modulos, generacion de tests, redaccion de documentacion tecnica y revision de coherencia entre componentes. Capacidad de edicion directa de archivos y ejecucion de comandos shell para validacion en tiempo real.
- **Codex de OpenAI**: utilizado en fases iniciales para generar el esqueleto del backend, dashboard y docker-compose. Documentado en commits anteriores a la rama actual.
- **ChatGPT (GPT-4)**: consultado puntualmente para resolver dudas conceptuales (calibracion isotonica, eleccion de buckets de histograma, patrones de Spark).

Justificacion de la combinacion: Claude Code excele en cambios extensos sobre el repositorio con validacion automatizada (ideal para la fase de consolidacion); Codex resulta mas agil para autocompletado puntual; ChatGPT es eficaz como pizarra conceptual. Las tres herramientas estan explicitamente listadas en el enunciado como aceptadas.

13.2 Ejemplos representativos de prompts

A continuacion se presentan seis prompts representativos del proceso de desarrollo, organizados por fase. Cada uno incluye el texto enviado a la herramienta, el resultado obtenido, la valoracion y el aprendizaje extraido.

Prompt 1 — Auditoria inicial del repositorio

Herramienta: Claude Code
Fase: Auditoria inicial

Prompt:

Revisa que todo lo del enunciado se cumple. Analiza el repositorio completo y dame un informe de que falta y que esta mal.

- **Resultado:** la IA identifico que el backend era un monolito en un solo archivo, que no habia tests, que el clasificador radiologico activo era un baseline estadistico sin modelo entrenado, y que faltaban pipeline Spark funcional, calidad de datos, automatizaciones y documentacion tecnica completa.
- **Valoracion:** acierto total. La auditoria fue precisa y accionable.
- **Aprendizaje:** dar a la IA acceso al enunciado junto con el repositorio permite detectar la brecha entre lo pedido y lo entregado mejor que una revision manual rapida.

Prompt 2 — Modularizacion del backend

Herramienta: Claude Code
Fase: Refactor arquitectonico

Prompt:

El backend esta en un solo archivo. Refactorizalo siguiendo una arquitectura en capas: api / services / repositories / db / storage. Cada capa solo puede importar la inmediatamente inferior. No rompas los tests que ya existen.

- **Resultado:** la IA propuso la estructura de directorios, los contratos de cada capa (interfaces implícitas via duck typing), el patron Unit of Work con context manager para transacciones ACID y la inyección de dependencias via *lru_cache* en *deps.py*. El refactor se completo sin romper ninguna ruta existente.
- **Valoracion:** acierto. La propuesta de Unit of Work fue espontanea (no estaba en el prompt) y resultado critica para la transaccionalidad.
- **Aprendizaje:** especificar restricciones de importacion en el prompt obliga a la IA a razonar sobre dependencias y genera arquitecturas mas limpias que pedir simplemente 'modulariza'.

Prompt 3 — Generacion de la suite de tests

Herramienta: Claude Code
Fase: Testing

Prompt:

Crea tests para el backend. No uses MagicMock generico: cada doble debe implementar la misma interfaz que la clase real. Los tests de API deben usar *dependency_overrides* de FastAPI, no parchear imports.

- **Resultado:** la IA genero 18 archivos de test (12 unit + 6 api) con fakes que implementan los contratos reales de *DatabaseSession* y *ObjectStorage*. Tres tests fallaron en la primera ejecucion por asunciones incorrectas sobre el comportamiento del codigo.
- **Valoracion:** iteracion necesaria. El codigo base tuvo que ajustarse en dos puntos menores (firma de metodos) para que los contratos fueran coherentes.
- **Aprendizaje:** la restriccion explicita de 'no MagicMock' en el prompt es imprescindible; sin ella la IA genera tests que pasan con cualquier implementacion, incluyendo las rotas.

Prompt 4 — Integracion del modelo de triaje

Herramienta: Claude Code
Fase: Integracion de modelos

Prompt:

El dashboard muestra 'Triage IA: fallback heuristico' aunque el archivo *triage_model.joblib* existe en */models*. Encuentra por que el backend no lo carga y arreglalo sin romper los tests.

- **Resultado:** la IA detecto que el archivo *triage_model.py* habia desaparecido en un merge (se verifiko con *git log --all --diff-filter=AD*). Lo recupero de un commit anterior, lo reintegro con el patron singleton thread-safe y anadio el bloque *services.triage_model* al schema *PlatformMetrics*, que era el motivo por el que el dashboard no recibia el estado real del modelo.
- **Valoracion:** acierto. La IA uso git para diagnosticar la causa raiz en lugar de suponer un error de configuracion.
- **Aprendizaje:** describir el sintoma observable (lo que muestra el dashboard) en lugar del problema tecnico subyacente permite a la IA rastrear la cadena causal completa.

Prompt 5 — Eliminacion de componentes no utilizados

Herramienta: Claude Code
Fase: Limpieza de infraestructura

Prompt:

Quita Prometheus y Grafana del proyecto. Desinstalalos, quita el codigo y asegurate de no romper nada por el camino.

- **Resultado:** la IA mapeo todas las referencias (12 archivos), elimino los contadores de *app/core/metrics.py* de los tres servicios que los importaban, borro el router *prometheus.py*, limpio el *docker-compose.yml*, elimino las carpetas de infraestructura, paro y borro los contenedores huérfanos y sus volúmenes, y valido con *docker compose config* antes de dar la tarea por completada.
- **Valoracion:** acierto. El orden de operaciones fue correcto: primero el código, luego la infraestructura, finalmente la validación.
- **Aprendizaje:** para tareas de eliminación es más seguro dejar que la IA mapee las dependencias antes de borrar; un borrado manual sin ese mapa habría dejado imports rotos.

Prompt 6 — Generación de la memoria técnica

Herramienta: Claude Code
Fase: Documentación

Prompt:

Quiero que hagas la memoria del proyecto. Haz TODA LA MEMORIA menos la parte de los prompts de la IA, esa dejala en TODO. Quiero un informe completo, dime el índice que propones y te digo. Luego hazlo en PDF, que se vea profesional y bien cuidado.

- **Resultado:** la IA propuso un índice de 16 capítulos + 3 anexos, lo valido con el equipo y genero un script Python con reportlab que produce un PDF de 43 páginas con portada, cabeceras, tablas, bloques de código y paleta de colores corporativa. Incorporo las métricas reales del CNN (leídas de *models/radiology_cnn_metrics.json*) y del Random Forest.
- **Valoracion:** acierto con una iteración. La primera versión incluía métricas del CNN como TODO (la IA no sabía que el modelo ya estaba entrenado); tras indicar que el compañero lo había entrenado y subido, la IA leyó el JSON de métricas y completo la sección.
- **Aprendizaje:** la IA no conoce el estado real del repositorio a menos que lo explore activamente; conviene indicar explícitamente que artefactos ya existen antes de pedir documentación.

13.3 Casos donde la IA acerto

- **Auditoria inicial del repositorio:** identificación precisa de puntos débiles (monolito en un solo archivo, ausencia de tests, baseline disfrazado de modelo entrenado).
- **Diseño transaccional:** propuesta espontánea del patrón Unit of Work al detectar inconsistencia entre operaciones de escritura separadas en distintas conexiones.
- **Carga perezosa y thread-safe del Random Forest:** singleton con *threading.Lock* que carga *triage_model.joblib* bajo demanda y respeta la concurrencia de FastAPI sin requerir estado global mutable.
- **Dobles de tests que respetan contratos reales:** rechazo explícito de MagicMock genérico en favor de fakes que implementan la interfaz.
- **Coherencia estilística entre módulos:** aplicación uniforme de patrones (config tipada, logging JSON, app factory) en backend, dashboard y pipeline.
- **Detección de contradicciones documentales:** identificación de inconsistencias entre README y memoria que un revisor exigente habría penalizado.

13.4 Casos donde hubo que corregir o iterar

- **Tests iniciales con asunciones incorrectas:** en la primera generacion, tres tests fallaron por suposiciones de la IA sobre el comportamiento del codigo. Se reescribieron los tests para reflejar el comportamiento real, no el esperado, sin modificar la logica de produccion.
- **Sobre-ingenieria espontanea:** la IA tendia a introducir middlewares, decoradores o abstracciones no justificadas. Cada propuesta de este tipo se rechazo y se exigió simplicidad.
- **Friccion entre response_model y dict literal:** una primera version de schemas Pydantic con alias *class* generaba conflictos al wirearlos como response_model. Se opto por aplicar response_model selectivamente.
- **Schemas Pydantic desalineados con la respuesta real:** una primera version del schema *PlatformMetrics* no incluia el bloque *services.triage_model*, lo que provocaba que el dashboard cayese a la rama de fallback. Se detecto al ver el dashboard mostrar *Triage IA: fallback heuristico* con el modelo cargado.
- **Templates obsoletos tras migracion a blueprints:** la IA detecto referencias a *url_for('logout')* en lugar de *url_for('auth.logout')* y propuso corregir los templates antes que retroceder.

13.5 Reflexion critica sobre el uso de IA

El uso de IA ha tenido un impacto sustancial en velocidad de implementacion y calidad estilistica del codigo, pero **determinadas tareas se mantuvieron deliberadamente en el ambito humano:**

- **Definicion de la arquitectura global:** la eleccion de capas, patrones y separacion de responsabilidades respondio a requisitos reales del enunciado, no se delego.
- **Criterios clinicos de alertas:** los thresholds (confianza < 0.55, deteccion COVID-19, banderas de calidad) son decisiones humanas con justificacion clinica.
- **Validacion clinica:** ningun modelo se considera apto para uso real sin revision medica.
- **Eleccion de stack tecnologico:** la decision PostgreSQL + MinIO + Spark fue del equipo; la IA documento los pros y contras.

Riesgos detectados: la IA tiende a la sobre-ingenieria sin control humano. Tambien puede ocultar errores sutiles si no se acompaña de tests. En este proyecto los tests han funcionado como red de seguridad y han detectado varios fallos en la primera ejecucion.

13.6 Estimacion del impacto en productividad

El trabajo de consolidacion del proyecto (refactor a arquitectura en capas, creacion de la suite de tests, integracion del modelo de triaje, configuracion de CI y redaccion documental) se ha realizado en aproximadamente **5-7 horas efectivas** de trabajo asistido por IA. Una estimacion conservadora del mismo trabajo manual (incluyendo investigacion de mejores practicas, escritura de tests y depuracion de la integracion entre servicios) se situaria entre 4 y 6 dias-persona. El factor de aceleracion estimado es del orden de **6 a 10x**.

Mas alla del ahorro de tiempo, el principal valor anadido fue la **coherencia estilistica entre modulos:** aplicar el mismo patron (config tipada, logging JSON, app factory, fakes con contrato) en backend, dashboard y pipeline habria sido tedioso manualmente y propenso a divergencias.

14. Reflexion critica

14.1 Limitaciones del sistema actual

- **CNN entrenado con dataset publico:** el ResNet18 alcanza 93.94 % de accuracy sobre test, pero el dataset proviene de fuentes publicas (no del propio hospital) y solo se han ejecutado 5 epochs; no constituye un modelo validado clinicamente.
- **Dataset radiografico no incluido:** el repositorio no contiene imagenes; se requiere descarga externa para entrenamiento (COVID-19 Radiography Database u otro).
- **Triaje con dataset sintetico:** el modelo se entrena sobre datos generados por reglas; un despliegue real requiere reentrenamiento con datos hospitalarios anonimizados.
- **Autenticacion local:** usuario/contrasena con hashes Werkzeug; sin integracion con SSO/LDAP/OIDC corporativo.
- **Sin HTTPS:** el prototipo expone HTTP plano; un despliegue real requiere reverse proxy con certificado de CA reconocida.
- **Sin cifrado en reposo:** PostgreSQL y MinIO operan sin encriptacion de disco ni de objetos.
- **Sin auditoria detallada:** se registran eventos de calidad pero no acciones individuales de usuarios.
- **Sin scheduler:** el pipeline se ejecuta on-demand; no hay disparo automatico ante nuevos ficheros.
- **Sin alta disponibilidad:** una sola instancia de cada servicio; no hay replicacion ni failover.
- **Solo procesamiento batch:** el pipeline procesa lotes; no hay ingesta streaming.

14.2 Posibles mejoras

- **Reentrenar el CNN con mas epochs y data augmentation extendida:** introducir scheduling del learning rate, mixup/cutmix y validacion cruzada para reducir la fluctuacion observada en la curva de validacion.
- **Caching de inferencia:** deduplicacion por checksum de imagen para evitar reprocesar la misma radiografia.
- **SSO corporativo:** migrar la autenticacion local a OAuth2 / OIDC con Keycloak o proveedor cloud.
- **Cifrado SSE-S3 en MinIO** con clave gestionada externamente (KMS).
- **Scheduler de pipeline:** Airflow o Prefect para disparar ingesta ante nuevos ficheros o por cron.
- **Trazabilidad distribuida:** OpenTelemetry para seguir una peticion entre dashboard, backend y pipeline.
- **Observabilidad externa:** el backend ya expone un snapshot agregado en `/metrics` consumido por el dashboard; un despliegue real anadiria un stack de metricas (Prometheus + Grafana, OpenTelemetry o equivalente) y agregacion de logs (ELK, Loki).
- **Tests de integracion end-to-end:** docker-compose dedicado a tests que levante PostgreSQL y MinIO efimeros antes de pytest.
- **Auditoria de seguridad OWASP ASVS:** rate limiting en `/predict` y `/triage`, validacion de Content-Length antes de leer cuerpo, revision de headers de seguridad.
- **Mejora de la clase *high* del triaje:** el recall 0.58 sugiere que las fronteras entre medium y high requieren reentrenamiento con datos reales y reajuste de los thresholds del MEWS/NEWS2 utilizados como base de etiquetado.

- **Internacionalizacion del dashboard:** actualmente solo en castellano; preparar i18n para catalan e ingles.

14.3 Viabilidad de aplicacion en un entorno real

Para llevar el sistema a produccion en un hospital real seria necesario, ademas de las mejoras tecnicas anteriores:

- **Validacion clinica del modelo CNN** por un equipo medico con dataset representativo.
- **Certificacion como producto sanitario:** clasificacion CE bajo el Reglamento (UE) 2017/745 (MDR) o equivalente FDA segun jurisdiccion. Probablemente clase IIa.
- **Auditoria de seguridad y privacidad** conforme al GDPR (Reglamento (UE) 2016/679) y normativa espanola de proteccion de datos sanitarios.
- **Acuerdo de tratamiento de datos** firmado con cualquier proveedor cloud que aloje datos clinicos.
- **Plan de respuesta ante incidentes** con notificacion en menos de 72 horas a la AEPD.
- **Plan de continuidad de negocio:** backups cifrados, plan de recuperacion ante desastres, replicacion en region secundaria.
- **Procedimientos operativos:** documentacion de altas/bajas de usuarios, gestion de claves, rotacion de contrasenas, revisiones periodicas de acceso.
- **Formacion del personal sanitario:** el sistema es una herramienta de apoyo, no un sustituto del criterio clinico.

15. Consideraciones éticas y legales

15.1 Sesgos en datos y modelos

Los modelos de clasificación de imágenes médicas son particularmente vulnerables a sesgos cuando el conjunto de entrenamiento no representa la diversidad clínica real de la población objetivo. Un modelo entrenado exclusivamente con radiografías procedentes de un único hospital, un único modelo de equipo radiológico o un rango demográfico estrecho puede exhibir un rendimiento significativamente inferior cuando se aplica fuera de ese contexto.

Riesgos específicos identificados

- **Sesgo de procedencia:** dataset dominado por un solo hospital, país o región.
- **Sesgo demográfico:** distribución no equilibrada por edad, sexo, etnia o nivel socioeconómico.
- **Sesgo de equipo radiológico:** imágenes capturadas con un fabricante o modelo concreto pueden engañar al modelo (aprende a clasificar por la marca de agua o características del equipo, no por la patología).
- **Sesgo temporal:** predominancia de imágenes de un periodo concreto (por ejemplo, primera ola de COVID-19) con características técnicas distintas a periodos posteriores.
- **Sesgo de etiquetado:** clasificaciones realizadas por un solo radiólogo, sin doble lectura ni consenso clínico.

Medidas propuestas

- **Transparencia del dataset:** documentar públicamente la composición (origen, equipos, periodos, demografía).
- **Partición por paciente:** separar train/val/test por paciente y no por imagen, para evitar fuga entre conjuntos.
- **Métricas estratificadas:** calcular precisión, recall y F1 por subgrupo demográfico y reportar la peor métrica como referencia.
- **Auditoría de la matriz de confusión:** especial atención a falsos negativos en patologías contagiosas.
- **Monitorización continua:** revisiones periódicas del rendimiento en producción con muestras representativas.

15.2 Riesgos en la decisión automatizada

El sistema **no está autorizado a tomar decisiones clínicas autónomas**. Su rol es de apoyo: ofrecer una clasificación preliminar, marcar casos para revisión prioritaria y reducir carga administrativa. La responsabilidad final del diagnóstico recae siempre en el personal sanitario.

Riesgos clínicos específicos

- **Falso negativo de COVID-19:** retraso en aislamiento, aumento del riesgo de contagio intrahospitalario. Es el error de mayor gravedad y debe ser objetivo explícito de minimización.
- **Falso negativo de neumonía:** retraso del tratamiento antibiótico y posible progresión clínica.
- **Falso positivo:** ansiedad del paciente, pruebas confirmatorias innecesarias, sobrecarga del circuito de revisión.

- **Automation bias:** tendencia humana a confiar mas en una recomendacion automatica de lo que la evidencia justifica.
- **Falso negativo en triaje critico:** paciente grave clasificado como menor riesgo; mitigado por el alto recall (0.92) en la clase *critical* y por la revision humana obligatoria.

Mitigaciones implementadas

- Cada prediccion se acompaña de una **nota clinica** explicita: *resultado de apoyo, no diagnostico*.
- Las predicciones con **confianza inferior a 0.55** generan un evento de baja confianza automaticamente.
- Toda prediccion **COVID-19** genera una alerta de severidad alta independientemente de la confianza.
- Las banderas de calidad de imagen (oscuridad, sobrexposicion, bajo contraste) se reportan al usuario para permitir repetir el estudio.
- El **historico completo** de estudios y triajes queda persistido para auditoria a posteriori.

15.3 Privacidad y proteccion de datos (RGPD)

Las imagenes medicas y los datos clinicos son **categorias especiales de datos personales** segun el articulo 9 del RGPD y la normativa espanola de proteccion de datos sanitarios. Este prototipo no contiene mecanismos suficientes para tratar datos reales de pacientes y debe usarse **exclusivamente con datos anonimizados, sinteticos o simulados**.

Carencias actuales del prototipo

- No hay cifrado en reposo de los objetos en MinIO ni de las filas en PostgreSQL.
- No hay auditoria detallada de accesos individuales por usuario.
- No hay gestion de consentimientos del paciente.
- Las copias de seguridad no estan automatizadas ni cifradas.
- No hay procedimiento documentado de borrado a peticion del interesado (derecho al olvido).

Medidas necesarias en un entorno real

- **Control de acceso por roles (RBAC)** con principio de minimo privilegio.
- **Cifrado en reposo:** PostgreSQL con disco cifrado (LUKS, BitLocker), MinIO con SSE-S3 o SSE-KMS.
- **Cifrado en transito:** TLS obligatorio en todos los flujos, incluyendo conexiones internas entre servicios.
- **Registro de auditoria:** cada acceso a un estudio queda registrado y es exportable para inspeccion.
- **Politica de retencion:** borrado seguro automatico al expirar el periodo legal de conservacion.
- **Minimizacion de datos:** almacenar solo lo estrictamente necesario para el proceso clinico.
- **Anonimizacion:** identificadores irreversiblemente anonimizados cuando se exporten datos para investigacion.
- **Acuerdo de tratamiento:** firma con cualquier proveedor cloud que aloje datos.
- **Plan de respuesta ante incidentes:** notificacion en menos de 72 horas a la AEPD ante cualquier brecha.

15.4 Responsabilidad clinica

La interpretacion final de cualquier estudio o triaje corresponde al personal sanitario. El sistema debe mostrar de forma visible explicaciones, niveles de incertidumbre, banderas de calidad y notas clinicas para evitar que los usuarios confundan una prediccion con un diagnostico definitivo. La trazabilidad se apoya en el registro completo de estudios y eventos en PostgreSQL y los informes JSON persistentes en MinIO.

15.5 Limitaciones legales del prototipo

El sistema descrito en esta memoria es un **prototipo academico**. El CNN ResNet18 alcanza un 93.94 % de accuracy sobre el conjunto de test publico, pero no constituye un modelo validado clinicamente: requiere validacion externa con datos hospitalarios reales, evaluacion formal por un equipo medico y, en su caso, certificacion regulatoria como producto sanitario antes de cualquier uso clinico real. El triaje opera con un modelo entrenado sobre datos sinteticos inspirados en MEWS/NEWS2; un despliegue real requiere reentrenamiento con datos hospitalarios y validacion frente a los protocolos vigentes.

16. Conclusiones

El proyecto ha cumplido los objetivos planteados en el enunciado: una plataforma integral de soporte hospitalario que combina IA y Big Data, desplegable con un unico comando, con dos modelos de IA operativos, pipeline Spark de procesamiento por lotes, almacenamiento dual estructurado y no estructurado, dashboard con autentificacion, automatizaciones de alertas y calidad, y documentacion profesional acompanada de tests automatizados.

16.1 Cumplimiento del enunciado

Requisito del enunciado	Estado
Modelo de IA justificado	Cumplido (RF triaje + CNN ResNet18)
Procesamiento Big Data con framework distribuido	Cumplido (Spark)
Dos tipos de almacenamiento combinados	Cumplido (PostgreSQL + MinIO)
Pipeline ingesta / limpieza / transformacion / analisis	Cumplido
Automatizacion de procesos	Cumplido (alertas, informes, bucket)
Visualizacion y comunicacion (dashboard)	Cumplido (Flask con auth)
Containerizacion con docker-compose	Cumplido (1 solo comando)
Calidad de datos y logging	Cumplido
Vibe Coding con herramienta IA	Cumplido (Claude Code + Codex)
Metodologia SDD	Cumplido (docs/SDD.md)
Clasificacion triple radiografias	Cumplido (Sana / Neumonia / COVID-19)
Matriz de confusion analizada	Cumplido (triaje y CNN sobre val y test)
Reflexion clinica del modelo	Cumplido
Consideraciones eticas y legales	Cumplido
Memoria tecnica con apartados obligatorios	Cumplido (este documento)
Diario de desarrollo con IA	Parcial (prompts pendientes)

16.2 Valor aportado

Mas alla del cumplimiento estricto del enunciado, el proyecto aporta valor en tres dimensiones:

- **Calidad de ingenieria:** arquitectura en capas con separacion estricta, transaccionalidad ACID, dobles de tests que respetan contratos, configuracion tipada fail-fast, logging estructurado.

- **Reproducibilidad:** cualquier persona puede levantar el sistema con *docker compose up -d* y reproducir el flujo end-to-end sin fricción. Los modelos se entrenan en perfiles dedicados.
- **Honestidad técnica:** el sistema documenta abiertamente sus limitaciones (CNN entrenado solo 5 epochs sobre dataset público, dataset sintético de triaje, ausencia de cifrado, ausencia de SSO) y reconoce que es un prototipo no apto para uso clínico real sin trabajo adicional.

16.3 Cierre

El sistema demuestra que es posible diseñar e implementar una plataforma hospitalaria realista en un entorno académico, equilibrando el alcance técnico con las consideraciones éticas y legales propias del sector sanitario. El stack tecnológico elegido (Python, FastAPI, Flask, PostgreSQL, MinIO, Spark, Docker, scikit-learn, PyTorch) es directamente transferible a un entorno profesional, y la arquitectura permite evolucionar a un despliegue real con cambios incrementales: añadir SSO, habilitar cifrado en reposo, incorporar un stack de observabilidad externo y certificar el modelo CNN clínicamente.

El proyecto se entrega como prototipo funcional end-to-end, documentado, testeado y reproducible, sentando una base sólida sobre la que un equipo real podría construir un producto sanitario completo.

0. Anexo A. Estructura del repositorio

```
Hospital_Project/
|- backend/                               FastAPI modular
|   |- app/
|   |   |- api/                           routers HTTP
|   |   |- services/                      logica de negocio
|   |   |- repositories/                  acceso a datos
|   |   |- db/                           session, pool, schema
|   |   |- storage/                       adaptador S3 (boto3)
|   |   |- schemas/                      DTOs Pydantic
|   |   |- core/                         config tipada, logging
|   |   '- utils/                        helpers transversales
|   |- tests/                            unit + api
|   |- Dockerfile
|   '- requirements.txt
|
|- dashboard/                             Flask con app factory
|   |- app/
|   |   |- factory.py
|   |   |- auth/
|   |   |- blueprints/                   /login, /, /upload, etc.
|   |   |- services/
|   |   |- presenters/
|   |   |- triage/
|   |   |- validation/
|   |   '- core/
|   |- templates/
|   |- static/
|   |- tests/                            unit + web
|   |- Dockerfile
|   '- requirements.txt
|
|- pipeline/                              Spark job modular
|   |- app/
|   |   |- core/                         config, logging, spark_factory
|   |   |- validators/                   csv_schema
|   |   |- transforms/                   quality_filter
|   |   |- writers/                      postgres_writer, minio_writer
|   |   '- reports/                      run_report
|   |- jobs/
|   |   '- radiology_pipeline.py
|   |- Dockerfile
|   '- requirements.txt
|
|- ml/                                     Entrenamiento de modelos
|   |- train_cnn.py                       ResNet18 para radiografias
|   |- train_triage.py                    Random Forest calibrado
|   |- model_contract.json
|   |- Dockerfile
|   '- requirements.txt
|
|- models/                                Artefactos
|   |- triage_model.joblib
|   '- triage_metrics.json
|
|- data/
|   |- incoming/                         CSV de ingesta
|   '- processed/                        Salida del pipeline
|
|- infrastructure/
```

```
|
|- docs/                Documentacion (este PDF, MD, SDD)
|- .github/workflows/   CI con GitHub Actions
|- .env.example
|- docker-compose.yml
|- iniciar_proyecto.bat
'- README.md
```

0. Anexo B. Variables de entorno

Las variables se cargan desde el archivo `.env`, que se construye a partir de `.env.example`. El archivo `.env` nunca se sube al repositorio.

PostgreSQL

```
POSTGRES_USER=hospital_user
POSTGRES_PASSWORD=<password fuerte>
POSTGRES_DB=hospital_db
```

MinIO

```
MINIO_ROOT_USER=hospital_minio_admin
MINIO_ROOT_PASSWORD=<password fuerte>
MINIO_REGION=us-east-1
MINIO_BUCKET_NAME=hospital-data
MINIO_AUTO_CREATE_BUCKET=true
MINIO_CONSOLE_URL=http://localhost:9001
```

Autenticacion del dashboard

```
SECRET_KEY=<clave aleatoria de al menos 32 bytes>
SESSION_COOKIE_SECURE=false
ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=<hash Werkzeug PBKDF2-SHA256>
USER_USERNAME=doctor
USER_PASSWORD_HASH=<hash Werkzeug PBKDF2-SHA256>
```

Pipeline

```
PIPELINE_INPUT_PATH=/data/incoming/radiology_studies.csv
PIPELINE_OUTPUT_PATH=/data/processed/radiology_clean
```

Entrenamiento CNN

```
TRAINING_EPOCHS=5
TRAINING_BATCH_SIZE=16
TRAINING_LEARNING_RATE=0.0005
TRAINING_PRETRAINED=false
```

Entrenamiento triaje

```
TRIAGE_SAMPLES=8000
TRIAGE_N_ESTIMATORS=400
TRIAGE_MAX_DEPTH=12
TRIAGE_NOISE_RATE=0.06
TRIAGE_RANDOM_STATE=42
```

0. Anexo C. Comandos de ejecucion

Arranque basico

```
cp .env.example .env          # Copiar variables
# Editar .env con credenciales reales
docker compose up -d          # Levantar entorno
docker compose ps              # Ver estado
docker compose logs -f         # Seguir logs
docker compose down            # Parar entorno
docker compose down -v         # Parar y borrar volúmenes
```

Ejecutar el pipeline Big Data

```
docker compose --profile pipeline up --build pipeline
```

Entrenar el modelo CNN de radiografías

```
# Requiere dataset en data/radiology_dataset/{train,val}/{Sana,Neumonia,COVID-19}/
docker compose --profile training run --rm model-trainer
```

Entrenar el modelo de triaje

```
docker compose --profile triage-training run --rm triage-trainer
```

Ejecucion de tests locales

```
# Backend
cd backend
pip install -r requirements-dev.txt
pytest --cov=app --cov-report=term

# Dashboard
cd ../dashboard
pip install -r requirements-dev.txt
pytest --cov=app --cov-report=term
```

Pruebas rapidas de API

```
# Salud del backend
curl http://localhost:8000/health

# Triage desde PowerShell
Invoke-RestMethod -Method Post -Uri "http://localhost:8000/triage" `
  -ContentType "application/json" `
  -Body '{"symptoms":["fever","shortness of breath"],"vitals":{"heart_rate":125,"oxygen_saturation":90,"systolic_bp":95}}\'

# Interfaces web
# http://localhost:8501      Dashboard
# http://localhost:8000/docs Swagger UI
# http://localhost:9001     Consola MinIO
```